# Swaraga: basic viva prep

Short answers to practise. You do not need to memorize the whole codebase.

**Start with this:** Our project explores Carnatic melody continuation. Given a short sequence of swaras, it predicts the next sequence. We clean the data, represent notes as scale degrees, and use raga rules to restrict the generated notes. The current demo uses an n-gram model.

### 1. What is the input and output?

For the main T2 task, the input is 16 musical positions and the output is the next 8 positions. Positions can include rest markers.

### 2. What is a raga? What is a tala?

A raga is a melodic framework involving notes, characteristic phrases and movement. A tala is a rhythmic cycle. An avartana is one complete cycle.

### 3. What is a swara? What is a gamaka?

A swara is a musical note category, such as S, R or G. A gamaka is an ornament or expressive pitch movement. Our reference-tone playback does not reproduce gamakas.

### 4. What dataset do we use?

The existing symbolic dataset contains 20 composition fragments, 20 ragas, 8 composers and 6 talas. It is very small, and its original notation sources are not independently verified.

### 5. Why do we preprocess the data?

To standardize names and notation, check errors and duplicates, and convert the data into consistent sequences. The original tokens had incorrect variants and missing rest positions, so they were rebuilt from notation.

### 6. What are scale degrees?

A scale degree is a note's position within the raga's scale. For Mohanam, S R2 G3 P D2 become 0 1 2 3 4. This lets models share relative melodic patterns across ragas.

## Remember these three words

**Training:** learning from examples. **Inference:** using the fitted model to predict. **Baseline:** a simple approach used for comparison.

Based on the local Swaraga implementation and submission notes, 27 October 2026. This is a revision sheet, not a new experimental report.

# Likely project questions

Keep each answer to two or three sentences.

### 7. Which models did you explore?

Random and direction-based baselines, an n-gram model, symbolic retrieval, a GRU and a Transformer. The current local swara demo uses the n-gram model; it does not train a neural network.

### 8. How does the n-gram model work?

It counts which notes tend to follow recent note patterns. It combines longer and shorter contexts so it can still predict when a long pattern is rare or unseen.

### 9. What is a GRU?

A Gated Recurrent Unit is a recurrent neural network that processes a sequence using a hidden state. Reset and update gates control how it uses previous information.

### 10. What is a Transformer?

A Transformer uses attention to combine information from relevant sequence positions. Our model uses a causal mask so it cannot look at future notes while predicting the next note.

### 11. How do you split the data? Why?

Whole compositions are split approximately 80% training, 10% validation and 10% test: 16, 2 and 2 here. Overlapping windows from the same composition stay together to avoid data leakage.

### 12. What is constrained decoding?

The model proposes next-note scores, and grammar rules restrict the available choices before selection. Scale-degree mapping keeps ordinary output notes in the chosen scale; being in-scale alone does not guarantee good music.

### 13. How do you evaluate the output?

We compare predicted notes with reference notes, compare melodic contours, and measure reference-token probability using perplexity. Listening is also needed because numerical scores do not fully capture musical quality.

### 14. What were the results and limitations?

In the saved T2 experiment, the n-gram had better contour distance and token accuracy than the GRU and Transformer. This is only a small-corpus result. We need more verified compositions and musical evaluation.

### 15. Does the demo generate a singer's voice?

No. Generated swaras use simple reference tones. The separate vocal feature selects existing recordings with manual annotations; it is not voice cloning. The current recording annotations are empty.

# Basic ML and PyTorch questions

These are the short definitions most useful for a viva.

### 16. What is PyTorch?

A Python framework for tensor computation and building neural networks. It provides layers, automatic differentiation and optimizers.

### 17. What is a tensor?

A multidimensional array. A scalar has zero dimensions, a vector has one, and a matrix has two. Tensors also have a data type and a device, such as CPU.

### 18. What is an embedding?

A learned vector representation of a discrete ID. An embedding converts note IDs into numerical feature vectors that a neural network can process.

### 19. What is a loss function?

It measures prediction error. We use cross-entropy for next-token prediction: assigning more probability to the correct note reduces the loss.

### 20. What are backpropagation and an optimizer?

Backpropagation computes how the loss changes with each parameter. An optimizer, such as Adam, uses those gradients to update the parameters.

### 21. What are an epoch, batch size and learning rate?

An epoch is one pass through the training data. Batch size is the number of examples in one update. Learning rate controls the scale of parameter updates.

### 22. What is overfitting? How can you reduce it?

Overfitting means learning training-specific patterns that do not generalize. More varied data, smaller models, dropout and early stopping can help.

### 23. What is teacher forcing?

During training, the model receives the true previous notes. During generation, it receives its own predictions, so errors can accumulate.

### 24. What is the difference between train(), eval() and no_grad()?

train() enables training behavior such as dropout. eval() switches such layers to evaluation behavior. no_grad() stops gradient recording; it does not automatically call eval().

**Most important sequence:** clear gradients → forward pass → calculate loss → backward pass → optimizer step.

# PyTorch syntax to practise

Read the comments, then try writing each example without looking.

## 1. Create tensors and inspect shapes

```
import torch
from torch import nn

x = torch.tensor([[1., 2.], [3., 4.]])
print(x.shape)                        # torch.Size([2, 2])
print(x[0])                           # tensor([1., 2.])
print(x[:, 1])                        # tensor([2., 4.])
print(x.reshape(4).shape)             # torch.Size([4])
zeros = torch.zeros(2, 3)             # 2 rows, 3 columns
random_values = torch.randn(2, 3)     # normal random values
```

## 2. Embedding and a linear layer

```
ids = torch.tensor([[1, 2, 3]], dtype=torch.long)
embedding = nn.Embedding(32, 8)       # 32 IDs, 8 features each
features = embedding(ids)
print(features.shape)                # [1, 3, 8]

layer = nn.Linear(8, 32)              # 8 features -> 32 scores
logits = layer(features)
print(logits.shape)                  # [1, 3, 32]
probs = torch.softmax(logits, dim=-1)
predicted_ids = logits.argmax(dim=-1) # shape [1, 3]
```

## 3. GRU syntax and saving weights

```
gru = nn.GRU(input_size=8, hidden_size=16, batch_first=True)
output, hidden = gru(features)
print(output.shape)                  # [1, 3, 16]
print(hidden.shape)                  # [1, 1, 16]

torch.save(gru.state_dict(), "gru_weights.pt")
```

**Know the arguments:** batch_first=True means input shape [batch, sequence, features]. dim=-1 means the last axis. argmax returns the highest-scoring index. softmax converts scores into probabilities. torch.long is used for discrete IDs.

**Common question:** Why no softmax before CrossEntropyLoss? It expects raw logits and handles the normalization internally. Use softmax when you want probabilities for inspection or sampling.

# The training loop to remember

A small standalone example. Synthetic labels demonstrate syntax, not project results.

```
import torch
from torch import nn

class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer = nn.Linear(4, 3)

    def forward(self, x):
        return self.layer(x)

model = Model()
x = torch.randn(5, 4)                # 5 examples, 4 features
y = torch.tensor([0, 1, 2, 1, 0])    # correct class IDs
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

model.train()
optimizer.zero_grad()                # 1. clear old gradients
logits = model(x)                    # 2. forward: shape [5,3]
loss = loss_fn(logits, y)            # 3. calculate error
loss.backward()                     # 4. calculate gradients
optimizer.step()                    # 5. update weights
print(loss.item())                   # Python number

model.eval()
with torch.no_grad():
    predictions = model(x).argmax(dim=1)
print(predictions)
```

## Be ready to explain these lines

**nn.Module:** base class for neural networks. **super().__init__():** initializes its machinery.
**forward():** defines how input becomes output. **model.parameters():** supplies parameters to the optimizer.
**zero_grad():** clears gradients because PyTorch accumulates them.
**backward():** computes gradients. **step():** changes weights. These are different operations.

**Last-minute priorities:** explain your project in 30 seconds; know why you split by composition; explain an n-gram, GRU and Transformer; write this training loop; trace [batch, time, features] shapes.

Sources: README.md; docs/SUBMISSION_NOTES.md; src/swaraga/models/neural.py; train.py; vocab.py; evaluate.py. PyTorch API references: Quickstart, GRU, CrossEntropyLoss.